

StyleSense : Generative AI–Powered Fashion Recommendation System

Project Description

StyleSense: Generative AI–Powered Fashion Recommendation System is an intelligent web application designed to provide personalized fashion advice and outfit recommendations using **Generative AI**. The platform analyzes user preferences, uploaded clothing images, skin tone, occasion, budget, and style choices to generate customized outfit suggestions that match individual tastes and current fashion trends.

The system combines modern web technologies with AI models to simplify the process of selecting stylish and coordinated outfits. Users can either request a **complete outfit recommendation** or **build an outfit around an existing clothing item they already own**. Based on the selected preferences, the AI recommends complementary apparel, footwear, and accessories while considering factors such as color harmony, seasonal suitability, and event appropriateness.

StyleSense also features an **AI-powered fashion chatbot** that answers style-related questions, provides wardrobe guidance, and assists users in making informed fashion decisions. To enhance the shopping experience, the platform generates curated **product recommendations with links to popular online retailers**, enabling users to explore and purchase similar items conveniently.

The application includes secure **user authentication, profile management, outfit history, and saved recommendations, allowing users to revisit previous styling sessions and maintain a personalized fashion journey**. By integrating image analysis, intelligent recommendation generation, and conversational AI, StyleSense offers a seamless and interactive experience for users seeking modern, data-driven fashion assistance.

Overall, **StyleSense** demonstrates how Generative AI can transform the fashion domain by delivering accessible, personalized, and context-aware styling solutions. It bridges the gap between technology and personal fashion, helping users make confident wardrobe choices while *reducing the time and effort involved in outfit planning*.

Scenarios

Scenario 1: Face Detection and Skin Tone Analysis

A user uploads a clear facial photograph to StyleSense for personalized styling assistance. The system performs face detection to identify the facial region and analyzes the user's skin tone using image processing techniques. Based on the **detected tone**, the AI suggests the most suitable color palettes, clothing combinations, and accessories, ensuring that future outfit recommendations complement the user's natural complexion.

Scenario 2: Complete Outfit Recommendation

A user logs into StyleSense and selects the “**Get Whole Outfit Styled**” option. They provide details such as gender, occasion, preferred style, budget, and color preferences. The AI analyzes these inputs along with the skin tone information and generates a complete outfit consisting of tops, bottoms, footwear, and accessories, accompanied by personalized styling tips and shopping recommendations.

Scenario 3: Build an Outfit Around an Existing Item

A user already owns a blue denim jacket and wants to create a coordinated look around it. They choose the “**Build Around My Item**” feature and specify the existing clothing item. StyleSense intelligently recommends matching shirts, trousers, shoes, and accessories, creating a balanced outfit while considering the user's preferences and budget.

Scenario 4: AI Fashion Chatbot Assistance

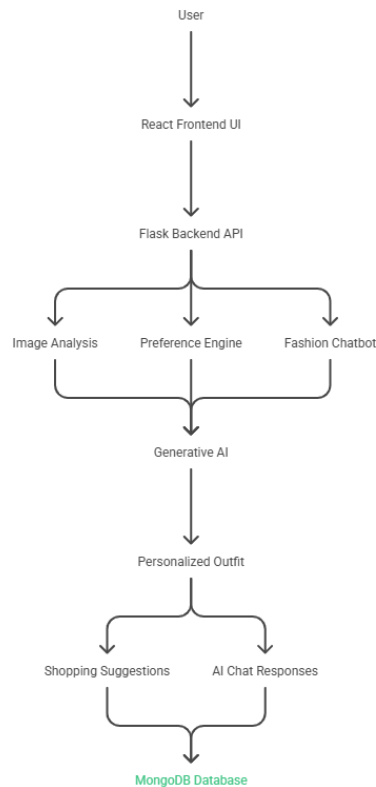
A user is unsure about what to wear for a traditional wedding ceremony and **asks the integrated AI Fashion Chatbot for guidance**. The chatbot provides personalized suggestions on appropriate attire, color combinations, accessories, and styling tips based on the occasion and current fashion trends, helping the user make informed fashion decisions.

Architecture Overview

StyleSense is a **Generative AI-powered fashion** recommendation platform built using a React frontend, Flask backend, MongoDB, and Groq's high-speed LLM inference API. The system combines face detection, skin tone analysis, and user preferences to generate personalized outfit recommendations and styling advice.

Its modular architecture supports features such as AI-powered outfit generation, fashion chatbot assistance, build-around-item recommendations, and shopping suggestions, while ensuring fast performance, scalability, and a seamless user experience.

Fashion AI Platform Architecture



Core Technologies

- **React.js:** Frontend library used to build a modern, responsive, and interactive user interface.
- **Flask:** Python web framework that handles backend logic, API routing, and request processing.
- **Groq API (LLaMA Models):** Powers the Generative AI features, including outfit recommendations and the fashion chatbot.
- **MongoDB:** NoSQL database used to store user accounts, chat history, saved outfits, and recommendation data.
- **MediaPipe:** Performs face detection and facial landmark extraction for accurate skin tone analysis.
- **OpenCV:** Processes uploaded images and supports computer vision tasks required for visual analysis.
- **JWT (JSON Web Tokens):** Provides secure user authentication and authorization.
- **HTML, CSS, and JavaScript:** Used with React to create a responsive and user-friendly interface.

Pre-requisites:

1. Python Environment Setup

Install Python 3.10 or later and create a virtual environment for the backend.

<https://www.python.org/downloads/>

2. Node.js Installation

Install Node.js 18+ (includes npm) for running the React frontend.

<https://nodejs.org/en/download>

3. Backend Dependencies

Install all required Python packages using:

pip install -r requirements.txt

https://pip.pypa.io/en/stable/cli/pip_install/

4. Frontend Dependencies

Install the required JavaScript packages using:

npm install

<https://docs.npmjs.com/cli/v9/commands/npm-install>

5. Database Setup

StyleSense uses MongoDB to store user accounts, chat history, saved outfits, and recommendation data.

- MongoDB Community Server: <https://www.mongodb.com/try/download/community>
- MongoDB Compass: <https://www.mongodb.com/products/tools/compass>

6. Backend Configuration

Create a .env file inside the backend directory and configure the required environment variables:

MONGO_URI=mongodb://localhost:27017/style sense

JWT_SECRET_KEY=your_secret_key

GROQ_API_KEY=your_groq_api_key

Environment variable management:

<https://pypi.org/project/python-dotenv/>

7. Computer Vision Dependencies

StyleSense uses MediaPipe and OpenCV for face detection and skin tone analysis.

Install the required packages:

pip install mediapipe

pip install opencv-python

OpenCV Documentation:

<https://opencv.org/>

MediaPipe Documentation:

<https://developers.google.com/mediapipe>

8. Frontend Configuration

Create a .env file inside the frontend directory with:

VITE_API_BASE_URL=http://localhost:5000

Vite Documentation:

<https://vitejs.dev/guide/>

9. AI Service Configuration

Generate a Groq API Key and add it to the backend .env file.

Groq Platform:

<https://console.groq.com/>

10. Backend Launch

Start the Flask backend server:

```
python app.py
```

Flask Documentation:

<https://flask.palletsprojects.com/>

11. Frontend Launch

Run the React development server:

```
npm run dev
```

<https://docs.npmjs.com/cli/v9/commands/npm-run-script>

12. Verification

After both servers start successfully:

- Backend API: http://localhost:5000
- Frontend Application: http://localhost:5173

Project Workflow

Milestone 1: Environment Setup and Project Initialization

- **Activity 1.1:** Create and configure the project structure for the React frontend and Flask backend.
- **Activity 1.2:** Set up a Python virtual environment and install backend dependencies.
- **Activity 1.3:** Install frontend packages using npm and configure the React environment.
- **Activity 1.4:** Configure .env files with **MongoDB**, **JWT**, and **Groq API** credentials.
- **Activity 1.5:** Run both backend and frontend servers to verify successful setup.

Milestone 2: Backend API Development

- **Activity 2.1:** Implement secure user authentication and authorization using JWT.
- **Activity 2.2:** Develop REST APIs for user profiles, outfit recommendations, chatbot, and history management.
- **Activity 2.3:** Implement image upload, validation, and processing functionality.
- **Activity 2.4:** Integrate **MediaPipe** and **OpenCV** for face detection and skin tone analysis.
- **Activity 2.5:** Connect MongoDB to store user data, chat history, and saved outfits.

Milestone 3: AI Integration and Recommendation Engine

- **Activity 3.1:** Integrate the **Groq LLM** to generate personalized outfit recommendations.
- **Activity 3.2:** Develop the **AI Fashion Chatbot** for fashion-related assistance.
- **Activity 3.3:** Implement the **Build Around My Item** recommendation feature.
- **Activity 3.4:** Generate personalized shopping suggestions based on user preferences and AI output.

Milestone 4: React Frontend Development

- **Activity 4.1:** Build responsive pages for authentication, home, and user profile.
- **Activity 4.2:** Develop interfaces for image upload, preference selection, and recommendation display.
- **Activity 4.3:** Implement chatbot, recommendation history, and saved outfits pages.
- **Activity 4.4:** Integrate frontend components with backend APIs for seamless interaction.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Perform functional testing of backend APIs and frontend components.
- **Activity 5.2:** Validate face detection, skin tone analysis, and AI-generated recommendations.
- **Activity 5.3:** Test end-to-end integration between the React frontend, Flask backend, MongoDB, and Groq API.
- **Activity 5.4:** Optimize performance, fix bugs, and deploy the application for production use.

Milestone 1: Environment Setup and Project Initialization

Activity 1.1: Create and configure the project structure for the React frontend and Flask backend.

In this activity, the project structure for the StyleSense application is created and organized to support both frontend and backend development. Separate directories are established for the React frontend and Flask backend to maintain modularity and ease of maintenance. Essential folders for components, routes, services, utilities, and database configurations are created to ensure a scalable architecture. This step provides a well-structured foundation for implementing application features and managing project resources efficiently.

```

└─ StyleSense
  └─ backend
    ├── __pycache__
    ├── middleware
    ├── models
    ├── routes
    ├── utils
    ├── venv
    ├── .env
    ├── app.py
    ├── db.py
    └── requirements.txt
  └─ frontend
    ├── dist
    ├── node_modules
    ├── public
    ├── src
    ├── .env
    ├── .gitignore
    ├── eslint.config.js
    ├── index.html
    ├── package-lock.json
    ├── package.json
    ├── README.md
    └── vite.config.js
```

Activity 1.2: Set up a Python virtual environment and install backend dependencies.

In this activity, a dedicated Python virtual environment is created using the `python -m venv venv` command to isolate backend dependencies from the global system environment. The

virtual environment is activated, and required backend packages such as Flask, Flask-CORS, PyMongo, OpenCV, Pillow, and Groq SDK are installed. This approach prevents dependency conflicts and ensures a stable development environment. This step establishes the backend workspace required for developing and testing server-side functionalities

```
PS D:\StyleSense> cd backend
PS D:\StyleSense\backend> python -m venv venv
```

```
PS D:\StyleSense\backend> venv\Scripts\activate
```

```
PS D:\StyleSense\backend> pip install flask flask-cors python-dotenv pymongo flask-jwt-extended groq bcrypt opencv-python numpy pillow scikit-learn
```

Activity 1.3: Install frontend packages using npm and configure the React environment.

In this activity, the React frontend environment is initialized, and all required frontend dependencies are installed using npm. Essential packages such as React Router, Axios, and UI-related libraries are added to support application development. The project configuration is verified to ensure proper compilation and execution of the React application. This step prepares the frontend environment for building interactive user interfaces and integrating backend services

```
PS D:\StyleSense> cd frontend
PS D:\StyleSense\frontend> npm install
```

Activity 1.4: Configure .env files with MongoDB, JWT, and Groq API credentials.

In this activity, environment configuration files are created to securely store application credentials and sensitive information. Database connection details for MongoDB, authentication settings for JWT, and API credentials for the Groq AI service are defined within the .env files. The application is configured to load these variables during runtime, ensuring secure access to external services without exposing confidential information in the source code. This step enhances application security and simplifies configuration management across development environments.

```
# Flask
SECRET_KEY=stylesense_flask_secret_key
# MongoDB
MONGO_URI=mongodb://localhost:27017/stylesenseai

# JWT
JWT_SECRET_KEY=myStyleSenseJWTSecret123

# Groq API
GROQ_API_KEY=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

# Upload Settings
UPLOAD_FOLDER=uploads
MAX_CONTENT_LENGTH=10485760

# Flask Environment
FLASK_ENV=development
FLASK_DEBUG=True
```

Activity 1.5: Run both backend and frontend servers to verify successful setup.

In this activity, the Flask backend server and React frontend development server are executed to validate the overall project setup. Initial tests are performed to ensure that the backend APIs are accessible and that the frontend application renders correctly in the browser. Communication between the frontend and backend is verified to confirm proper integration and functionality. This step confirms that the development environment is correctly configured and ready for implementing the core features of the StyleSense application.

```
PS D:\StyleSense\frontend> npm run dev

> frontend@0.0.0 dev
> vite

Port 5173 is in use, trying another one...

VITE v8.0.16 ready in 570 ms

→ Local:   http://localhost:5174/
→ Network: use --host to expose
→ press h + enter to show help
```

```
PS D:\StyleSense\backend> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
```

Milestone 2: Backend API Development

Activity 2.1: Implement secure user authentication and authorization using JWT.

In this activity, a secure authentication system is implemented to manage user registration and login functionalities within the StyleSense application. JSON Web Tokens (JWT) are used to authenticate users and protect restricted API endpoints from unauthorized access. During login, valid user credentials are verified, and a JWT token is generated and returned to the client. This token-based authentication mechanism ensures secure communication between the frontend and backend while maintaining user session integrity throughout the application.

```
@auth_bp.route("/signup", methods=["POST"])
def signup():
    data = request.get_json()
    email = data.get("email", "").strip().lower()
    password = data.get("password", "")
    name = data.get("name", "").strip()

    if not email or not password or not name:
        return jsonify({"error": "Email, password, and name are required"}), 400

    if len(password) < 6:
        return jsonify({"error": "Password must be at least 6 characters"}), 400

    result = create_user(email, password, name)

    if "error" in result:
        return jsonify(result), 409

    user = result["user"]
    token = create_access_token(identity=str(user["_id"]))

    return jsonify({
        "message": "Account created successfully",
        "token": token,
        "user": {
            "id": user["_id"],
            "name": user["name"],
            "email": user["email"],
            "profile": user.get("profile", {})
        }
    }), 201

@auth_bp.route("/login", methods=["POST"])
def login():
    data = request.get_json()
    email = data.get("email", "").strip().lower()
    password = data.get("password", "")

    if not email or not password:
        return jsonify({"error": "Email and password are required"}), 400

    result = verify_user(email, password)

    if "error" in result:
        return jsonify(result), 401

    user = result["user"]
    token = create_access_token(identity=str(user["_id"]))

    return jsonify({
        "message": "Login successful",
        "token": token,
        "user": {
            "id": user["_id"],
            "name": user["name"],
            "email": user["email"],
            "profile": user.get("profile", {})
        }
    }), 200
```

Activity 2.2: Develop REST APIs for user profiles, outfit recommendations, chatbot, and history management.

In this activity, RESTful API endpoints are developed to support the core functionalities of the StyleSense application. APIs are created for managing user profiles, generating personalized

outfit recommendations, interacting with the AI-powered chatbot, and retrieving recommendation history. Each endpoint is designed to handle specific user requests and return structured responses for seamless frontend integration. This step establishes the communication layer between the user interface, AI services, and database components.

```
# JWT Configuration
app.config["JWT_SECRET_KEY"] = os.getenv("JWT_SECRET_KEY", "dev-secret-key")
app.config["JWT_ACCESS_TOKEN_EXPIRES"] = False
jwt = JWTManager(app)

# Register Blueprints
app.register_blueprint(auth_bp, url_prefix="/api/auth")
app.register_blueprint(photo_bp, url_prefix="/api/photo")
app.register_blueprint(outfit_bp, url_prefix="/api/outfit")
app.register_blueprint(profile_bp, url_prefix="/api/profile")
app.register_blueprint(chatbot_bp, url_prefix="/api/chat")
app.register_blueprint(history_bp, url_prefix="/api/history")

@app.route("/api/health")
def health():
    return {"status": "ok", "message": "StyleSense AI is running"}
```

```
result = generate_build_around_recommendations(
    gender=gender,
    occasion=occasion,
    budget=budget,
    base_item_name=base_item_name,
    selected_categories=selected_categories,
    budget_allocations=allocations,
    detected_color=detected_color,
    detected_pattern=detected_pattern,
    age=age,
    color_palette=color_palette,
    skin_tone=skin_tone
)
```

```
return jsonify({
    "session_id": rec_id,
    "skin_tone": skin_tone,
    "outfits": outfits,
    "color_palette": color_palette,
    "overall_advice": ai_data.get("overall_advice", ""),
    "generated_for": {
        "gender": gender,
        "occasion": occasion,
        "age": age,
        "budget": budget
    }
}), 200
```

Activity 2.3: Implement image upload, validation, and processing functionality.

In this activity, functionality is developed to allow users to upload facial images for skin tone analysis. Uploaded images are validated based on supported file formats, size restrictions, and image quality requirements before being processed. The backend securely stores and manages uploaded files while preparing them for further analysis. This step ensures that only valid images are accepted, improving the reliability and accuracy of the recommendation system.

```
@photo_bp.route("/analyze", methods=["POST"])
@jwt_required_custom
def analyze_photo():
    """Analyze uploaded photo for skin tone detection."""
    user_id = get_current_user_id()
    data = request.get_json()

    image_data = data.get("image")
    if not image_data:
        return jsonify({"error": "No image provided"}), 400

    # Detect skin tone
    result = detect_skin_tone(image_data)

    if "error" in result and not result.get("detected"):
        return jsonify({
            "error": "Could not detect face in image. Please upload a clear photo.",
            "details": result.get("error")
        }), 422

    skin_tone = result["skin_tone"]
    skin_hex = result.get("hex", "#b48c6e")

    # Save to user profile
    update_skin_tone(user_id, skin_tone, skin_hex)

    # Get color palette recommendations
    palette = get_color_palette(skin_tone)

    return jsonify({
        "skin_tone": skin_tone,
        "skin_hex": skin_hex,
        "rgb": result.get("rgb"),
        "color_palette": palette,
        "message": f"Skin tone detected: {skin_tone}"
    }), 200
```

Activity 2.4: Integrate MediaPipe and OpenCV for face detection and skin tone analysis.

In this activity, MediaPipe and OpenCV are integrated to perform facial detection and skin tone analysis on uploaded user images. MediaPipe is utilized to accurately identify facial landmarks and regions of interest, while OpenCV processes the detected facial area to extract skin color information. The analyzed skin tone is then categorized and used as the basis for generating personalized fashion and color recommendations. This step forms the core intelligence behind the StyleSense recommendation engine.

```
with mp_face_mesh.FaceMesh(
    static_image_mode=True,
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5
) as face_mesh:
    results = face_mesh.process(img_rgb)

    if not results.multi_face_landmarks:
        # Fallback: sample center region of image
        return _fallback_skin_detection(img_rgb)

    face_landmarks = results.multi_face_landmarks[0]
    skin_pixels = []

    for idx in FACE_SKIN_LANDMARKS:
        if idx < len(face_landmarks.landmark):
            lm = face_landmarks.landmark[idx]
            x = min(int(lm.x * w), w - 1)
            y = min(int(lm.y * h), h - 1)
            pixel = img_rgb[y, x]
            skin_pixels.append(pixel)

    if not skin_pixels:
        return _fallback_skin_detection(img_rgb)

    skin_pixels = np.array(skin_pixels, dtype=np.float32)
    avg_color = skin_pixels.mean(axis=0)
    r, g, b = int(avg_color[0]), int(avg_color[1]), int(avg_color[2])

    skin_tone = classify_skin_tone(r, g, b)
```

```
def classify_skin_tone(r: int, g: int, b: int) -> str:
    """
    Classify skin tone based on RGB values using ITA (Individual Typology Angle).
    ITA = arctan((L* - 50) / b*) * 180 / pi
    """
    # Convert to LAB for better skin tone analysis
    rgb_pixel = np.uint8([[r, g, b]])
    lab_pixel = cv2.cvtColor(rgb_pixel, cv2.COLOR_RGB2LAB)[0][0]
    L, a, b_val = float(lab_pixel[0]), float(lab_pixel[1]), float(lab_pixel[2])

    # Normalize LAB values
    L_norm = L * 100.0 / 255.0
    b_norm = b_val - 128.0

    # ITA angle calculation
    if b_norm == 0:
        b_norm = 0.001
    ita = np.arctan((L_norm - 50) / b_norm) * (180 / np.pi)

    # Classification based on ITA ranges
    if ita > 55:
        return "Fair"
    elif ita > 41:
        return "Light"
    elif ita > 28:
        return "Medium"
    elif ita > 10:
        return "Olive"
    elif ita > -30:
        return "Tan"
    else:
        return "Deep"
```

Activity 2.5: Connect MongoDB to store user data, chat history, and saved outfits

In this activity, MongoDB is integrated with the Flask backend to provide persistent storage for application data. Collections are designed to store user account information, chatbot interactions, recommendation history, skin tone analysis results, and saved outfit preferences. Database operations for creating, retrieving, updating, and managing records are implemented to support application functionality. This step ensures efficient data management and enables personalized recommendations based on user activity and preferences.

```
def create_user(email: str, password: str, name: str) -> dict:
    """Create a new user with hashed password."""
    users = get_collection("users")

    if users.find_one({"email": email}):
        return {"error": "Email already registered"}

    hashed = bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt())

    user = {
        "email": email,
        "password": hashed,
        "name": name,
        "profile": {
            "gender": None,
            "occasion": None,
            "age": None,
            "budget": 2000,
            "skin_tone": None,
            "skin_hex": None
        },
        "saved_outfits": [],
        "past_preferences": [],
        "created_at": datetime.utcnow()
    }

    result = users.insert_one(user)
    user["_id"] = str(result.inserted_id)
    user.pop("password", None)
    return {"user": user}
```

```
def save_outfit(user_id: str, outfit: dict) -> bool:
    """Save an outfit to user's favorites."""
    users = get_collection("users")
    try:
        outfit["saved_at"] = datetime.utcnow().isoformat()
        result = users.update_one(
            {"_id": ObjectId(user_id)},
            {"$push": {"saved_outfits": outfit}}
        )
        return result.modified_count > 0
    except Exception:
        return False

def get_saved_outfits(user_id: str) -> list:
    """Get user's saved outfits."""
    user = get_user_by_id(user_id)
    if user:
        return user.get("saved_outfits", [])
    return []
```

```
def add_past_preference(user_id: str, preference: str) -> bool:
    """Add a style preference to user history (max 20 kept)."""
    users = get_collection("users")
    try:
        users.update_one(
            {"_id": ObjectId(user_id)},
            {
                "$push": {
                    "past_preferences": {
                        "$each": [preference],
                        "$slice": -20
                    }
                }
            }
        )
        return True
    except Exception:
        return False
```

```
from pymongo import MongoClient
from dotenv import load_dotenv
import os

load_dotenv()

_client = None
_db = None

def get_db():
    global _client, _db
    if _db is None:
        mongo_uri = os.getenv("MONGO_URI", "mongodb://localhost:27017/style sense")
        _client = MongoClient(mongo_uri)
        _db = _client.get_database()
    return _db

def get_collection(name: str):
    return get_db()[name]
```

Milestone 3: AI Integration and Recommendation Engine

Activity 3.1: Integrate the Groq LLM to generate personalized outfit recommendations.

In this activity, the Groq Large Language Model (LLM) is integrated into the StyleSense application to generate personalized outfit recommendations. User-specific attributes such as skin tone, gender, age, occasion, budget, and past preferences are provided as inputs to the AI model through carefully designed prompts. Based on these inputs, the model generates suitable outfit combinations, color suggestions, styling advice, and fashion recommendations. This integration enables the system to provide intelligent and personalized fashion guidance tailored to individual user requirements.

```
from groq import Groq
from dotenv import load_dotenv
import os
import json
import re
from urllib.parse import quote, quote_plus

load_dotenv()

client = Groq(api_key=os.getenv("GROQ_API_KEY"))

def _call_groq_with_fallback(messages, temperature=0.7, max_tokens=1500, response_format=None):
    """Call Groq API with prioritized fallback models to handle rate limits or outages."""
    models = ["llama-3.3-70b-versatile", "llama-3.1-8b-instant", "qwen/qwen3-32b"]
    last_err = None
    for model in models:
        try:
            print(f"Attempting Groq API call with model: {model}")
            kwargs = {
                "model": model,
                "messages": messages,
                "temperature": temperature,
                "max_tokens": max_tokens
            }
            if response_format:
                kwargs["response_format"] = response_format
            response = client.chat.completions.create(**kwargs)
            return response
        except Exception as e:
            last_err = e
            print(f"Groq API call failed for model {model}: {e}")
            continue
    raise last_err if last_err else Exception("No Groq models succeeded")
```

Activity 3.2: Develop the AI Fashion Chatbot for fashion-related assistance.

In this activity, an AI-powered fashion chatbot is developed using the Groq LLM to provide

users with interactive fashion assistance. The chatbot is designed to answer queries related to clothing styles, color combinations, seasonal trends, outfit matching, and fashion recommendations. User messages are processed through the AI model, and context-aware responses are generated in real time. This feature enhances user engagement by offering personalized fashion guidance and styling support through a conversational interface.

```
@chatbot_bp.route("/message", methods=["POST"])
@jwt_required_custom
def chat():
    """Handle fashion chatbot message with validation and persistence."""
    user_id = get_current_user_id()
    data = request.get_json()

    message = data.get("message", "").strip()
    # History from request can be passed or we can query DB
    conversation_history = data.get("history", [])

    if not message:
        return jsonify({"error": "Message cannot be empty"}), 400

    if len(message) > 500:
        return jsonify({"error": "Message too long (max 500 characters)"}), 400

    chats_col = get_collection("chats")

    # 1. Save User Message to DB
    user_chat_doc = {
        "userId": user_id,
        "role": "user",
        "message": message,
        "timestamp": datetime.utcnow()
    }
    chats_col.insert_one(user_chat_doc)

    # 2. Before-Sending Validation
    if not is_fashion_related(message):
        # Save Assistant Out-of-Scope Response to DB
        assistant_chat_doc = {
            "userId": user_id,
            "role": "assistant",
            "message": OUT_OF_SCOPE_RESPONSE,
            "timestamp": datetime.utcnow()
        }
        chats_col.insert_one(assistant_chat_doc)
        return jsonify({
            "response": OUT_OF_SCOPE_RESPONSE,
            "message": message
        }), 200
```

Activity 3.3: Implement the Build Around My Item recommendation feature.

In this activity, the "Build Around My Item" feature is implemented to help users create complete outfits based on an existing clothing item. Users can upload an image or specify a clothing item they already own, and the system analyzes its characteristics such as color, pattern, and category. Using AI-generated recommendations, complementary clothing items,

footwear, and accessories are suggested to create a coordinated outfit. This feature allows users to maximize the use of their existing wardrobe while receiving personalized styling suggestions.

```
if data.get("mode") == "build_around":
    from utils.image_analyzer import analyze_clothing_image, allocate_budgets
    from utils.groq_ai import generate_build_around_recommendations

    selected_categories = data.get("selected_categories") or []
    base_item_name = data.get("base_item_name") or ""
    base_item_image = data.get("base_item_image") or None

    # Smart budget allocation
    allocations = allocate_budgets(budget, selected_categories)

    # Analyze image for color and pattern
    detected_color = None
    detected_pattern = None
    if base_item_image:
        analysis = analyze_clothing_image(base_item_image)
        if analysis.get("success"):
            detected_color = analysis.get("color_name")
            detected_pattern = analysis.get("pattern")

    # Call groq for build_around mode
    result = generate_build_around_recommendations(
        gender=gender,
        occasion=occasion,
        budget=budget,
        base_item_name=base_item_name,
        selected_categories=selected_categories,
        budget_allocations=allocations,
        detected_color=detected_color,
        detected_pattern=detected_pattern,
        age=age,
        color_palette=color_palette,
        skin_tone=skin_tone
    )
```

Activity 3.4: Generate personalized shopping suggestions based on user preferences and AI output.

In this activity, personalized shopping suggestions are generated by combining user preferences, skin tone analysis results, and AI-generated outfit recommendations. The system creates relevant product search queries and provides shopping links for recommended clothing items, footwear, and accessories. Recommendations are filtered according to the user's budget, occasion, and style preferences to ensure practical and relevant suggestions. This feature assists users in discovering fashion products that align with their personalized style recommendations and purchasing requirements.

```
def generate_product_search_queries(outfit: dict) -> list:
    """Extract shopping search queries from an outfit recommendation."""
    queries = []
    if outfit.get("top"):
        queries.append({
            "part": "Top",
            "query": outfit["top"].get("search_query", ""),
            "budget": outfit["top"].get("estimated_cost")
        })
    if outfit.get("bottom"):
        queries.append({
            "part": "Bottom",
            "query": outfit["bottom"].get("search_query", ""),
            "budget": outfit["bottom"].get("estimated_cost")
        })
    if outfit.get("footwear"):
        queries.append({
            "part": "Footwear",
            "query": outfit["footwear"].get("search_query", ""),
            "budget": outfit["footwear"].get("estimated_cost")
        })
    for acc in outfit.get("accessories", []):
        queries.append({
            "part": "Accessory",
            "query": acc.get("search_query", ""),
            "budget": acc.get("estimated_cost"),
            "item_name": acc.get("item", "")
        })
    return queries
```

```
def build_shopping_links(search_query: str, gender: str = None, occasion: str = None, budget: int = None) -> dict:
    """Build direct shopping links for major Indian e-commerce platforms with gender, occasion, and budget context."""
    if not search_query:
        return {}

    search_query = search_query.strip()
    contextual_terms = [search_query]

    gender_term = _normalize_gender_search_term(gender)
    if gender_term and gender_term not in search_query.lower():
        contextual_terms.append(gender_term)

    if occasion:
        occasion_text = occasion.strip().lower()
        if occasion_text not in search_query.lower():
            contextual_terms.append(f"for {occasion_text}")

    if budget:
        budget_text = f"under ₹{budget}"
        if budget_text not in search_query.lower():
            contextual_terms.append(budget_text)

    contextual_query = " ".join(contextual_terms)
    encoded = quote_plus(contextual_query)
    myntra_path = quote(contextual_query.replace(" ", "-"), safe="-")

    return {
        "amazon": f"https://www.amazon.in/s?k={encoded}",
        "myntra": f"https://www.myntra.com/{myntra_path}",
        "ajio": f"https://www.ajio.com/search/?text={encoded}",
        "flipkart": f"https://www.flipkart.com/search?q={encoded}"
    }
```

Milestone 4: React Frontend Development

Activity 4.1: Build responsive pages for authentication, home, and user profile.

In this activity, responsive frontend pages are developed using React.js to provide an intuitive and user-friendly interface for the StyleSense application. Authentication pages are created to support user registration and login, while the home page serves as the central hub for accessing application features. A user profile page is also implemented to display personal information, skin tone analysis results, preferences, and saved recommendations. Responsive design principles are applied to ensure compatibility across different devices and screen sizes, enhancing the overall user experience.

```
function PrivateRoute({ children }) {
  const { user, loading } = useAuth();
  if (loading) return <div className="loading-screen"><div className="spinner" /></div>;
  return user ? <Navigate to="/login" />;
}

function PublicRoute({ children }) {
  const { user } = useAuth();
  return user ? <Navigate to="/dashboard" /> : children;
}

function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Toaster
          position="top-right"
          toastOptions={{
            style: { background: "#1a1a2e", color: "#e0e0f0", border: "1px solid #7c3aed" }
          }}
        />
        <Routes>
          <Route path="/" element={<LandingPage />} />
          <Route path="/login" element={<PublicRoute><LoginPage /></PublicRoute>} />
          <Route path="/signup" element={<PublicRoute><SignupPage /></PublicRoute>} />
          <Route path="/dashboard" element={<PrivateRoute><DashboardPage /></PrivateRoute>} />
          <Route path="/style" element={<PrivateRoute><StylingPage /></PrivateRoute>} />
          <Route path="/saved" element={<PrivateRoute><SavedOutfitsPage /></PrivateRoute>} />
          <Route path="/history" element={<PrivateRoute><HistoryPage /></PrivateRoute>} />
          <Route path="/profile" element={<PrivateRoute><ProfilePage /></PrivateRoute>} />
          <Route path="/chat" element={<PrivateRoute><ChatPage /></PrivateRoute>} />
          <Route path="*" element={<Navigate to="/" />} />
        </Routes>
      </BrowserRouter>
    </AuthProvider>
  );
}

export default App;
```

Activity 4.2: Develop interfaces for image upload, preference selection, and recommendation display.

In this activity, interactive user interfaces are developed to support the core functionality of the StyleSense application. An image upload module is created to allow users to submit facial images for skin tone analysis. Preference selection forms are implemented to collect information such as gender, occasion, age, and budget, which are required for generating personalized recommendations. Dedicated recommendation display pages are designed to present AI-generated outfit suggestions, color palettes, styling advice, and shopping recommendations in a visually appealing and organized manner.

```
const analyzeImage = async (base64) => {
  setAnalyzing(true);
  try {
    const res = await photoAPI.analyze(base64);
    onDetected(res.data);
    toast.success(`Skin tone detected: ${res.data.skin_tone}`);
  } catch (err) {
    toast.error(err.response?.data?.error || "Could not analyze image");
  } finally {
    setAnalyzing(false);
  }
};

const handleFile = (file) => {
  if (!file.type.startsWith("image/")) {
    toast.error("Please upload an image file");
    return;
  }
  const reader = new FileReader();
  reader.onload = (e) => {
    setPreview(e.target.result);
    analyzeImage(e.target.result);
  };
  reader.readAsDataURL(file);
};

const handleDrop = (e) => {
  e.preventDefault();
  setDragOver(false);
  const file = e.dataTransfer.files[0];
  if (file) handleFile(file);
};

const captureWebcam = useCallback(() => {
  const imageSrc = webcamRef.current.getScreenshot();
  if (imageSrc) {
    setPreview(imageSrc);
    setMode("upload");
    analyzeImage(imageSrc);
  }
}, [webcamRef]);
```

```
function ShopLinks({ links }) {
  if (!links) return null;
  return (
    <div className="shop-links">
      {Object.entries(links).map(([store, url]) => (
        <a
          key={store}
          href={url}
          target="_blank"
          rel="noopener noreferrer"
          className={`shop-link ${SHOP_CLASSES[store] || ""}`}
        >
          <ShoppingBag size={10} />
          {store.charAt(0).toUpperCase() + store.slice(1)}
        </a>
      ))}
    </div>
  );
}

export default function OutfitCard({ outfit, showSave = true }) {
  const [expanded, setExpanded] = useState(false);
  const [saved, setSaved] = useState(false);
  const [saving, setSaving] = useState(false);

  const handleSave = async () => {
    setSaving(true);
    try {
      await outfitAPI.save(outfit);
      setSaved(true);
      toast.success("Outfit saved!");
    } catch {
      toast.error("Could not save outfit");
    } finally {
      setSaving(false);
    }
  };
};
```

Activity 4.3: Implement chatbot, recommendation history, and saved outfits pages.

In this, additional frontend modules are developed to improve user engagement and personalization. An AI Fashion Chatbot interface is implemented to provide real-time fashion

assistance and styling guidance through conversational interactions. A recommendation history page is created to allow users to revisit previously generated outfit suggestions, while a saved outfits page enables users to store and manage their favorite recommendations. These features enhance usability by providing easy access to personalized fashion information and previously generated content.

```
const sendMessage = async (text) => {
  const msg = text || input.trim();
  if (!msg) return;

  setInput("");
  const userMsg = { role: "user", content: msg };
  setMessages((prev) => [...prev, userMsg]);
  setLoading(true);

  // Build history for context (exclude the initial greeting)
  const history = messages.slice(1).map((m) => ({
    role: m.role === "bot" ? "assistant" : "user",
    content: m.content,
  }));

  try {
    const res = await chatAPI.message(msg, history);
    setMessages((prev) => [...prev, { role: "bot", content: res.data.response }]);
  } catch {
    setMessages((prev) => [
      ...prev,
      { role: "bot", content: "Sorry, I'm having trouble right now. Please try again!" },
    ]);
  } finally {
    setLoading(false);
  }
};
```

```
<div className="chat-container">
  {/* Messages */}
  <div className="chat-messages">
    {messages.map((msg, i) => (
      <div key={i} className={`chat-message ${msg.role}`}>
        {msg.role === "bot" && (
          <div className="chat-avatar" style={{ background: "rgba(124,58,237,0.2)" }}>
            <Sparkles size={16} style={{ color: "var(--accent-light)" }} />
          </div>
        )}
        <div className="chat-bubble" style={{ whiteSpace: "pre-wrap" }}>
          {msg.content}
        </div>
      </div>
    ))}
```

```
export default function HistoryPage() {
  const [history, setHistory] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const load = async () => {
      try {
        const res = await historyAPI.getAll(30);
        setHistory(res.data.history || []);
      } catch {
        toast.error("Could not load history");
      } finally {
        setLoading(false);
      }
    };
    load();
  }, []);
}
```

```
{expanded && (
  <div className="fade-in" style={{ marginTop: 20 }}>
    {rec.overall_advice && (
      <div style={{
        padding: "12px 16px",
        background: "rgba(212,168,67,0.08)",
        borderRadius: "var(--radius)",
        marginBottom: 16,
        fontSize: "0.85rem",
        color: "var(--text-secondary)",
        borderLeft: "3px solid var(--gold)"
      }}>
        {rec.overall_advice}
      </div>
    )}
    {rec.color_palette && (
      <ColorPalette
        palette={rec.color_palette}
        skinTone={rec.skin_tone}
        skinHex={rec.skin_hex}
      />
    )}
    <div style={{ display: "flex", flexDirection: "column", gap: 16, marginTop: 16 }}>
      {rec.outfits?.map((outfit, i) => (
        <OutfitCard key={i} outfit={outfit} />
      ))}
    </div>
  </div>
)};
);
}
```

Activity 4.4: Integrate frontend components with backend APIs for seamless interaction.

In this activity, frontend components are integrated with backend REST APIs to enable smooth communication between the user interface and server-side services. Axios is utilized to send requests and receive responses for authentication, image uploads, skin tone analysis, outfit

recommendation generation, chatbot interactions, and recommendation history retrieval. Dynamic data rendering is implemented to display real-time information received from the backend. This integration ensures seamless functionality across all application modules and delivers a responsive user experience throughout the StyleSense platform.

```
const API_BASE = import.meta.env.VITE_API_URL || "http://localhost:5000/api";
const api = axios.create({
  baseURL: API_BASE,
  headers: { "Content-Type": "application/json" },
});

// Attach JWT token to every request
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("stylesense_token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Redirect to login on 401
api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      localStorage.removeItem("stylesense_token");
      localStorage.removeItem("stylesense_user");
      window.location.href = "/login";
    }
    return Promise.reject(err);
  }
);
```

```
export const outfitAPI = {
  generate: (data) => api.post("/outfit/generate", data),
  save: (outfit) => api.post("/outfit/save", { outfit }),
  getSaved: () => api.get("/outfit/saved"),
  deleteSaved: (name) => api.delete(`~outfit/saved/${encodeURIComponent(name)}`),
};
```

Milestone 5: Testing and Deployment

Activity 5.1: Perform functional testing of backend APIs and frontend components.

In this activity, comprehensive functional testing is conducted on both the Flask backend APIs and React frontend components of the StyleSense application. Backend endpoints related to user authentication, image upload, outfit recommendation generation, chatbot interactions, profile management, and recommendation history are tested to ensure correct request handling and response generation. Frontend components, including login forms, image upload interfaces, recommendation displays, chatbot pages, and profile modules, are validated to ensure proper rendering and user interaction. This testing process helps verify that all application features operate according to the specified requirements.

Activity 5.2: Validate face detection, skin tone analysis, and AI-generated recommendations.

In this activity, the core intelligent features of the StyleSense application are tested and validated. Uploaded facial images are processed using MediaPipe and OpenCV to verify accurate face detection and skin tone extraction. Different sample images are evaluated to ensure consistency in skin tone classification and color palette generation. Additionally, AI-generated outfit recommendations produced through the Groq LLM are reviewed to confirm their relevance to user attributes such as skin tone, gender, occasion, age, and budget. This validation process ensures the reliability and effectiveness of the recommendation engine.

Activity 5.3: Test end-to-end integration between the React frontend, Flask backend, MongoDB, and Groq API.

In this activity, end-to-end integration testing is performed to verify seamless communication among all major components of the StyleSense application. User actions initiated from the React frontend, such as registration, image upload, recommendation requests, and chatbot interactions, are traced through the Flask backend, MongoDB database, and Groq AI services. Data flow, API responses, database operations, and recommendation generation processes are thoroughly validated to ensure that all modules work together correctly. This testing confirms the stability and reliability of the complete system workflow.

Activity 5.4: Optimize performance, fix bugs, and deploy the application for production use.

In this activity, the overall performance of the StyleSense application is optimized to improve responsiveness and user experience. Issues identified during testing are resolved, including frontend rendering problems, API response delays, validation errors, and recommendation inconsistencies. Database queries and image-processing operations are refined to enhance execution efficiency. After successful optimization and bug fixing, the application is prepared for deployment by configuring production settings, environment variables, and security parameters. This step ensures that the application is stable, secure, and ready for real-world usage.

LANDING PAGE:


Sign In
Get Started

Next-Gen AI Styling

Discover Your Perfect Style with AI

Get personalized outfit recommendations, color analysis, style guidance, and fashion insights powered by artificial intelligence.

Get Started
Sign In

98%
MATCH ACCURACY

10k+
OUTFITS CURATED

4
TOP INDIAN BRANDS



Premium styling
Casual Chic
Smart Casual

Explore Style Categories

No matter the occasion, StyleSense has you covered with custom curated sets.



Casual

Everyday comfort with effortless, cool styles for coffee runs or weekend hangouts.



Formal

Sharp, professional, and tailored looks that command respect in boardrooms.



Ethnic

Traditional Indian wear with modern elegance for weddings, festivals, and family events.



Sports

High-performance athleisure and activewear combining mobility with contemporary aesthetics.



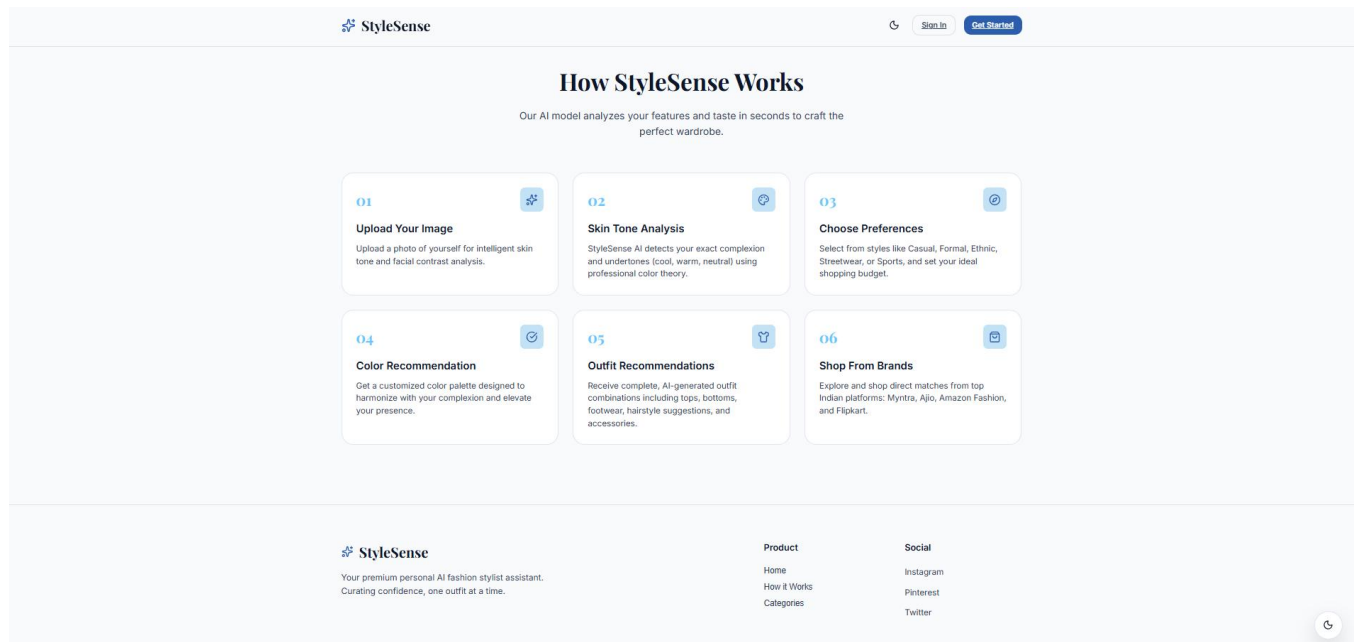
Streetwear

Edgy, bold designs inspired by urban culture, featuring oversized hoodies and sneakers.

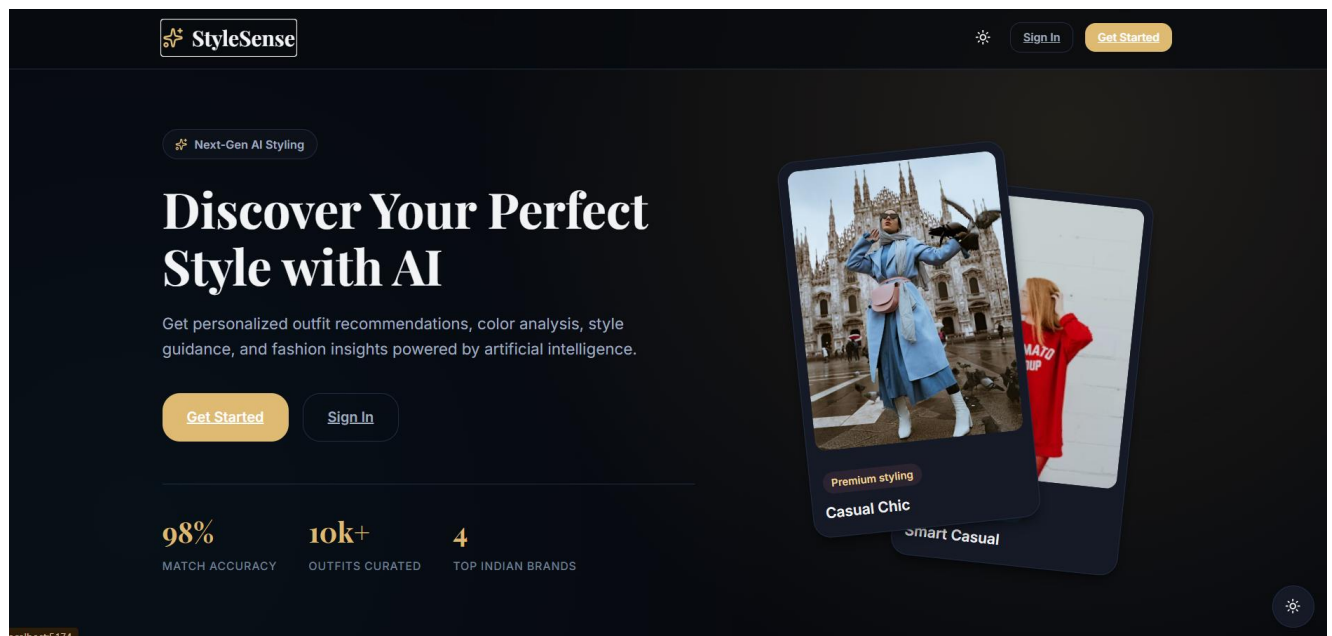


Party Wear

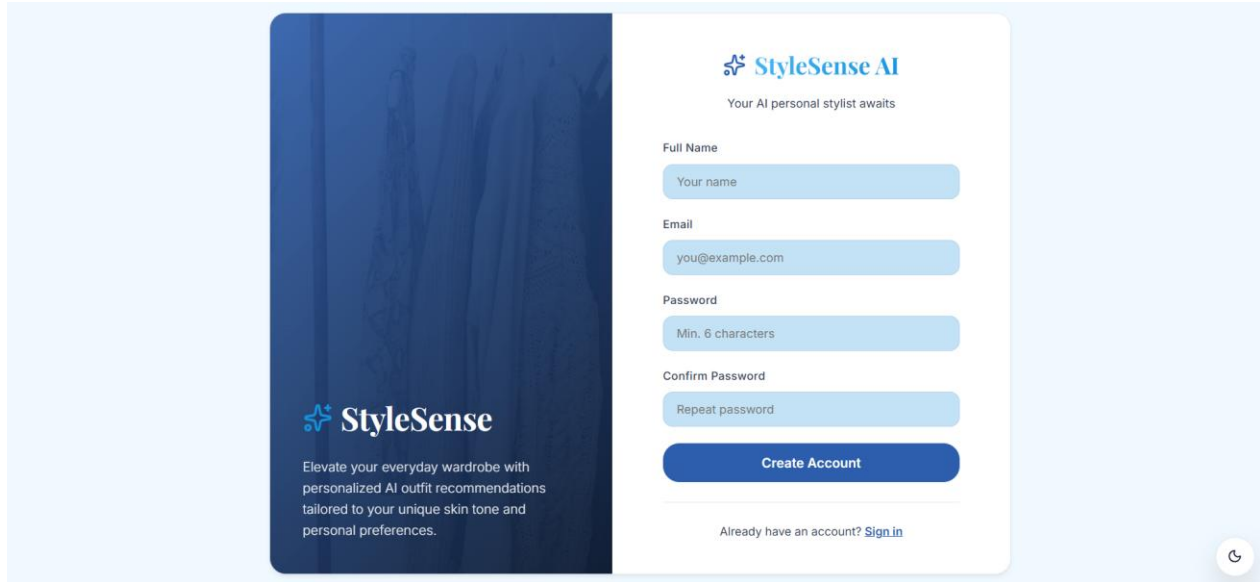
Showstopping evening ensembles, cocktail dresses, and blazers to light up any night.



DARK THEME:



USER SIGNUP PAGE:



The User Signup page features a dark blue sidebar on the left with the StyleSense logo and a description: "Elevate your everyday wardrobe with personalized AI outfit recommendations tailored to your unique skin tone and personal preferences." The main content area is white and contains the StyleSense AI logo and the text "Your AI personal stylist awaits". Below this, there are four input fields: "Full Name" (placeholder: "Your name"), "Email" (placeholder: "you@example.com"), "Password" (placeholder: "Min. 6 characters"), and "Confirm Password" (placeholder: "Repeat password"). A blue "Create Account" button is positioned below the input fields. At the bottom, there is a link: "Already have an account? [Sign In](#)". A small circular icon with a right arrow is located in the bottom right corner.

StyleSense AI
Your AI personal stylist awaits

Full Name
Your name

Email
you@example.com

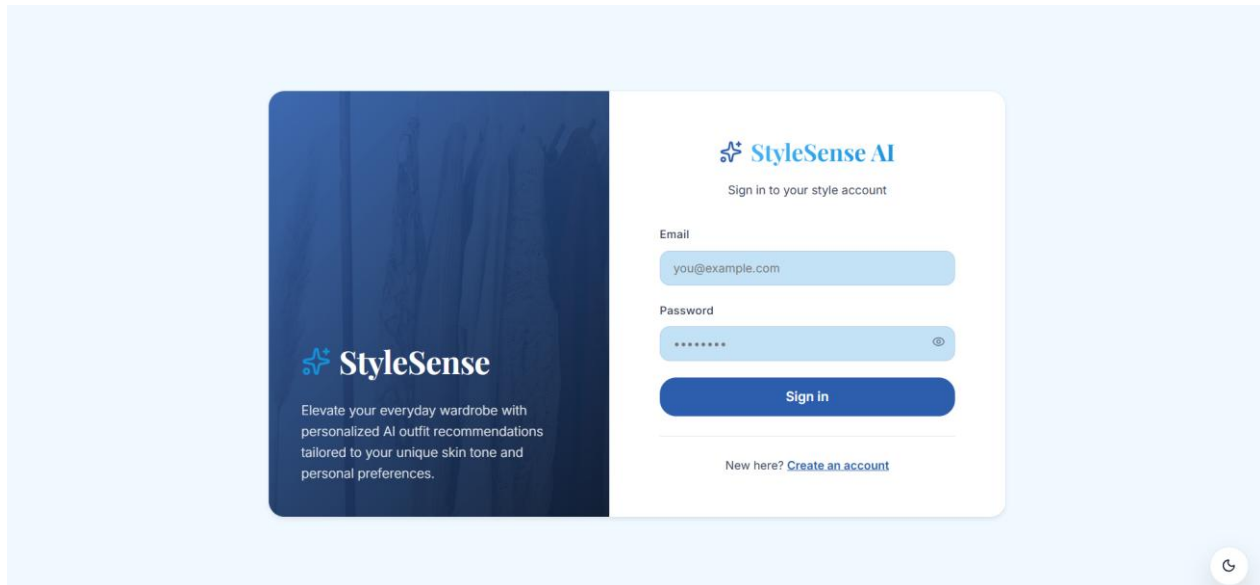
Password
Min. 6 characters

Confirm Password
Repeat password

Create Account

Already have an account? [Sign In](#)

USER LOGIN PAGE:



The User Login page features a dark blue sidebar on the left with the StyleSense logo and a description: "Elevate your everyday wardrobe with personalized AI outfit recommendations tailored to your unique skin tone and personal preferences." The main content area is white and contains the StyleSense AI logo and the text "Sign in to your style account". Below this, there are two input fields: "Email" (placeholder: "you@example.com") and "Password" (placeholder: "*****" with an eye icon). A blue "Sign in" button is positioned below the input fields. At the bottom, there is a link: "New here? [Create an account](#)". A small circular icon with a right arrow is located in the bottom right corner.

StyleSense AI
Sign in to your style account

Email
you@example.com

Password

Sign in

New here? [Create an account](#)

DASHBOARD:

StyleSense

AI Fashion Stylist

Dashboard

Get Styled

Saved Outfits

History

Style Chat

Profile

Test user

Sign out

Good Morning, Test! 🌞

Welcome! Complete your style profile and let our AI curate outfit recommendations for you.

Generate New Outfit

BEST COLORS

● ● ●

Navy · Cobalt · Sky

AGE

Not set

Complete profile

STYLE SCORE

60/100

Complete profile

OUTFITS GENERATED

0

Keep exploring!

Your Style Moodboard



+ Style Inspired Look

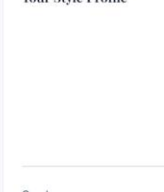


+ Style Inspired Look



+ Style Inspired Look

Personalized Vibe



Your Style Profile

Edit



Test user

Not Set Tone

Generate

UPLOAD IMAGE FOR SKINTONE ANALYSIS:

StyleSense

AI Fashion Stylist

Dashboard

Get Styled

Saved Outfits

History

Style Chat

Profile

Test user

Sign out

Get Styled

Upload your photo and let AI craft the perfect complete outfit for you.

Get Whole Outfit Styled

Build Around My Item

Step 1 of 3: Photo & Skin Tone

Upload Your Photo

We'll analyze your skin tone to find colors and styles that complement you.

Upload Photo

Use Camera

Drop a photo here or click to browse

Best results: clear facial photo, good lighting

Color Suggestions

Your AI color palette will appear here after you upload a photo.

● ● ● ●

Upload a Photo

FACE DETECTION & SKIN TONE ANALYSIS RESULT:

StyleSense
AI Fashion Stylist

- Dashboard
- Get Styled**
- Saved Outfits
- History
- Style Chat
- Profile

Test user
[Sign out](#)

Get Styled

Upload your photo and let AI craft the perfect complete outfit for you.

[Get Whole Outfit Styled](#)
[Build Around My Item](#)

Step 1 of 3: Photo & Skin Tone

Upload Your Photo

We'll analyze your skin tone to find colors and styles that complement you.

[Upload Photo](#)
[Use Camera](#)


[Retake Photo](#)

[Continue to Preferences >](#)

Your Color Palette — Fair Tone

Soft, cool, and muted tones complement fair skin beautifully.

PRIMARY COLORS

#C8A2C8
#7B9EA8
#E8B4B8
#A8B5C8

SECONDARY / NEUTRALS

#F5E6D3
#D4E6C3
#E8D5C4
#C8D4E0

ACCENT COLORS

#B84513
#2E4A6B
#6B4226
#4A6741

Colors to avoid

Skin tone detected: Fair

FASHION PREFERENCES:

StyleSense
AI Fashion Stylist

- Dashboard
- Get Styled**
- Saved Outfits
- History
- Style Chat
- Profile

Test user
[Sign out](#)

Step 2 of 3: Your Preferences

Your Style Preferences

Gender

[Male](#)
[Female](#)
[Non-binary](#)
[Prefer not to say](#)

Occasion

[Casual](#)
[College](#)
[Office](#)
[Party](#)
[Formal](#)
[Wedding](#)

Age

Budget (₹)

[₹500](#)
[₹1,000](#)
[₹2,000](#)
[₹5,000](#)
[₹10,000](#)
[₹20,000](#)

[Back](#)
[Generate Recommendations](#)



Style Inspiration

"Fashion is part of the daily air and it changes all the time, with all the events. You can even see the approaching of a revolution in clothes."

Anna Wintour

PERSONALIZED OUTFIT RECOMMENDATION PAGE:

StyleSense

AI Fashion Stylist

Dashboard

Get Styled

Saved Outfits

History

Style Chat

Profile

Get Styled

Upload your photo and let AI craft the perfect complete outfit for you.

Get Whole Outfit Styled Build Around My Item

Step 3 of 3: Outfit Recommendations

View Color Palette Upload Image

Style Advice: For a fair skin tone, it's essential to balance cool and warm tones in the outfit. Consider adding a pop of color with accessories or a bold lip color to enhance the overall look. Don't be afraid to experiment with different textures and fabrics to add depth to the outfit.

recommended outfits for you

Elegant Formal

Balanced Estimated Cost: ₹4,000

This outfit suits her fair skin tone and formal occasion, with a mix of Rose Dust and Slate Gray creating a harmonious palette. The combination of a tailored top and trousers is both modern and versatile, fitting for a 20-year-old.

Top

Formal silk shirt — Rose Dust — Silk

ajio amazon flipkart

Bottom

Tailored trousers — Slate Gray — Cotton Blend

ajio amazon flipkart

Footwear

Low-heeled pumps — Neutral

ajio amazon flipkart

More details

Modern Chic

Balanced Estimated Cost: ₹3,300

This outfit is perfect for a young adult with a fair skin tone, looking for a modern and chic formal look. The combination of Lilac and Steel Blue creates a striking yet harmonious palette.

Top

Peplum top — Lilac — Chiffon

ajio amazon flipkart

Bottom

Wide-leg pants — Steel Blue — Cotton

ajio amazon flipkart

Footwear

Kitten heels — Neutral

ajio amazon flipkart

More details

Sophisticated Elegance

Premium Estimated Cost: ₹4,900

This outfit exudes sophistication and elegance, making it perfect for a formal occasion. The combination of Slate Gray and Rose Dust creates a stunning and harmonious palette that complements the fair skin tone.

Top

Elegant silk shirt — Rose Dust — Silk

ajio amazon flipkart

Bottom

Tailored pencil skirt — Slate Gray — Wool Blend

ajio amazon flipkart

Footwear

High-heeled pumps — Neutral

ajio amazon flipkart

More details

New Session

SHOPPING SUGGESTION PAGES:



MEN WOMEN KIDS HOME BEAUTY GENZ STUDIO NEW

Search for products, brands and more

Profile Wishlist Bag

Moda Rapido (3)

Kinjo (3)

StyleCast x Revolve (2)

+ 21 more

PRICE

₹200 - ₹6,800+

COLOR

Green (18)

Pink (12)

Blue (9)

Cream (7)

Teal (6)

Purple (6)

Peach (5)

+ 15 more

DISCOUNT RANGE

20% and above

30% and above

40% and above

50% and above



4.3 ★ | 240

DressBerry

Women Printed Satin Top

Rs. 519 ~~Rs. 1999~~ (Rs. 1480 OFF)



4.3 ★ | 44

Jolene.club

V-Neck Crepe Top

Rs. 1426 ~~Rs. 1899~~ (25% OFF)

Only Few Left!



4.3 ★ | 44

Amagyya

Mandarin Collar Blouson Top

Rs. 549 ~~Rs. 2499~~ (78% OFF)

Only Few Left!



4.3 ★ | 44

OFFICE & YOU

Green Crepe Top Drape Sho...

Rs. 2700

Only Few Left!



4.3 ★ | 44

PURYS WORKSPACE

Women Puff Sleeves Top

Rs. 494 ~~Rs. 1999~~ (75% OFF)



4.3 ★ | 240

DressBerry

Women Printed Satin Top

Rs. 519 ~~Rs. 1999~~ (Rs. 1480 OFF)



4.3 ★ | 44

Jolene.club

V-Neck Crepe Top

Rs. 1426 ~~Rs. 1899~~ (25% OFF)

Only Few Left!



4.3 ★ | 44

Amagyya

Mandarin Collar Blouson Top

Rs. 549 ~~Rs. 2499~~ (78% OFF)

Only Few Left!



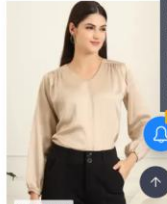
4.3 ★ | 44

OFFICE & YOU

Green Crepe Top Drape Sho...

Rs. 2700

Only Few Left!



4.3 ★ | 44

PURYS WORKSPACE

Women Puff Sleeves Top

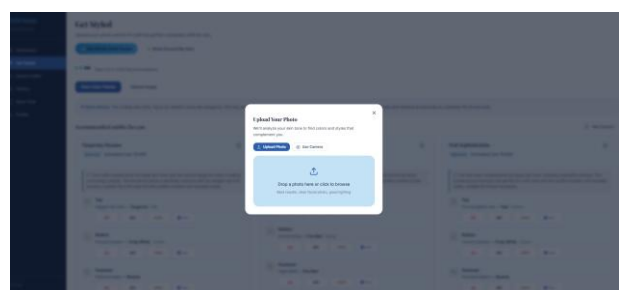
Rs. 494 ~~Rs. 1999~~ (75% OFF)

UPTO ₹500 OFF

<https://www.myntra.com/tops/dream-beauty-fashion/dream-beauty-fashion-puff-sleeve-top/42409551/buy>

Dante Exam Tutor

Trending now



BUILD AROUND MY ITEM FEATURE:

StyleSense

AI Fashion Stylist

Dashboard

Get Styled

Saved Outfits

History

Style Chat

Profile

Get Styled

Tell AI about an item you own to build a complete matching look around it.

Get Whole Outfit Styled

Build Around My Item

Step 1 of 3: Your Base Item

Upload or Describe Your Clothing Item

Upload an image of your item (optional) or describe it manually below.

Upload Photo

Use Camera

✓ Item Image Uploaded Successfully!

Your item is displayed in full resolution on the right side.

Remove Image

Change Image

Retake Photo

Item Description / Name

Brown Shirt

Specify the color, style, or fabric if skipping photo upload.

Gender (To display correct matching categories)

Male

Female

Non-binary

Prefer not to say

Continue to Pairings

STYLE & PAIRING PREFERENCES:

Get Styled

Tell AI about an item you own to build a complete matching look around it.

Get Whole Outfit Styled
 Build Around My Item

Step 2 of 3: Categories & Budget

Style & Pairing Preferences

Select Categories to Recommend
Budget will automatically distribute across the categories you select.

☐ Shirts/T-shirts	☒ Pants/Jeans	☐ Kurtas	☐ Sherwanis	☐ Nehru Jackets	☐ Ethnic Jackets
☐ Salwars	☐ Chudidars	☒ Shoes	☒ Watch	☐ Belt	☐ Bag
☐ Sunglasses	☐ Accessories				

Occasion

Casual
College
Office
Party
Formal
Wedding

Age

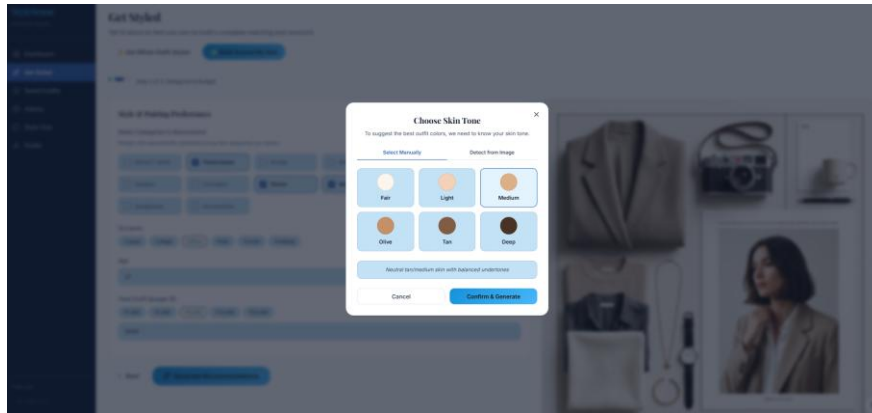
21

Total Outfit Budget (₹)

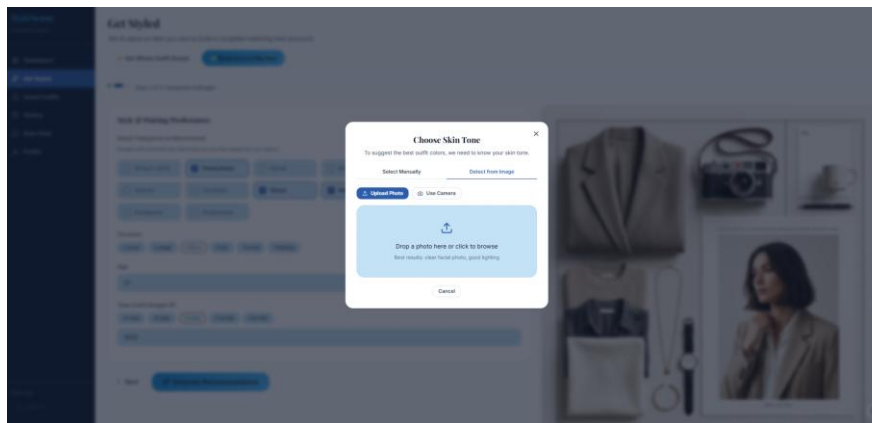
₹1,000
₹2,000
₹5,000
₹10,000
₹20,000

< Back
 Generate Recommendations

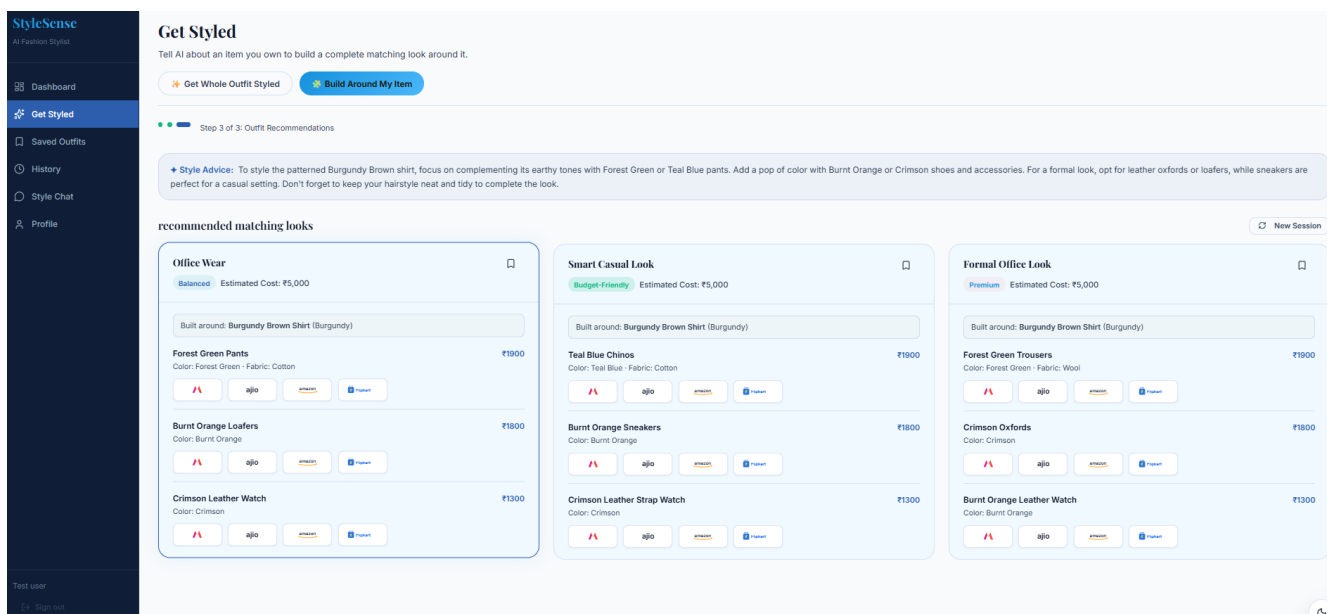
SKINTONE SELECTION:



SKINTONE ANALYSIS USING IMAGE:



REQUIRED ITEMS RECOMMENDATION PAGE:



SHOPPING SUGGESTION PAGES:



MEN WOMEN KIDS HOME BEAUTY GENZ STUDIO **NEW**

Search for products, brands and more

Profile Wishlist Bag

FILTERS

Bundles Country of Origin Size

Sort by: Recommended

CATEGORIES

☐ Trousers (72191)

☐ Rain Trousers (11)

BRAND

☐ Roadster (2385)

☐ StyleCast x Revote (2268)

☐ Suo (1879)

☐ StyleCast (1816)

☐ V-Mart (1438)

☐ Allen Solly (1396)

☐ Snitch (1376)

☐ Van Heusen (1367)

+ 956 more

PRICE

₹200 - ₹10,200+

COLOR

☐ Grey (13702)

☐ Black (13692)

NEW

Van Heusen
Men Slim Fit Formal Trousers
Rs. 1207 ~~Rs. 2625~~ (54% OFF)
Only Few Left!

Blackberrys
Men Chinos Trousers
Rs. 1221 ~~Rs. 2599~~ (53% OFF)
Only Few Left!

Raymond
Men Slim Fit Formal Trousers
Rs. 878 ~~Rs. 2199~~ (60% OFF)

Park Avenue
Men Striped Formal Trousers
Rs. 1010 ~~Rs. 2199~~ (54% OFF)

H&M
Relaxed Fit Cotton Chinos
Rs. 1259 ~~Rs. 2299~~ (45% OFF)

UPTO ₹500 OFF

TIME

₹690 - ₹1,650+

Up to ₹900

₹900 - ₹1,200

₹1,200 - ₹1,400

Over ₹1,400

Deals & Discounts

All Discounts

Coupons

Customer Reviews

★★★★☆ & Up

Discount

10% Off or more

25% Off or more

35% Off or more

50% Off or more

60% Off or more

70% Off or more

Shoe Material

☐ Plastic

☐ Polyurethane (PU)

☐ Faux Leather

Special Features

☐ Lightweight

Results

Check each product page for other buying options. Price and other details may vary based on product size and colour.

Centrino
Men's Formal Shoes | Lace-up Oxford Shoe with Solid Desig...
Top Reviewed for Comfort
3.8 ★★★★★ (7.5K)
Limited time deal
₹799 M.R.P.: ₹1,299 (56% off)
Up to 5% back with Amazon Pay IC...
FREE delivery Sat, 13 Jun

Centrino
Derby Formal Shoes for Men | Classic Lace-Up Dress Shoes |...
Top Reviewed for Comfort
3.8 ★★★★★ (22K)
Limited time deal
₹749 M.R.P.: ₹1,299 (66% off)
Up to 5% back with Amazon Pay IC...
FREE delivery Sat, 13 Jun

BXXY
Men's Casual Formal Slipon Moccasin and Loafer Outdoo...
₹2,299 M.R.P.: ₹4,999 (54% off)
Up to 5% back with Amazon Pay IC...
FREE delivery Wed, 17 Jun
Add to cart

Monte Carlo
Men's Solid Genuine Leather Monk Shoes (201816FW)
3.8 ★★★★★ (81)
₹3,348 M.R.P.: ₹5,199 (56% off)
Save 3% with coupon (limited sizes/colours)
FREE delivery Sat, 13 Jun
Or fastest delivery Tomorrow, 12

FASHION VICTIM
Men's Textured Synthetic Leather Loafers | Lightweight...
5.0 ★★★★★ (1)
₹1,119 M.R.P.: ₹2,499 (55% off)
Save 2% with coupon (limited sizes/colours)
FREE delivery Wed, 17 Jun
Add to cart



MEN WOMEN KIDS HOME BEAUTY GENZ STUDIO **NEW**

Search for products, brands and more

Profile Wishlist Bag

FILTERS

Bundles Country of Origin Size

Sort by: Recommended

CATEGORIES

☐ Watches (1749)

☐ Watch Straps (232)

BRAND

☐ TALGO (239)

☐ BLACK LOVIES (230)

☐ Miss Perfect (212)

☐ CSAMON (194)

☐ VIRAKTI (186)

☐ WROGN (163)

☐ WATCHSTAR (154)

☐ IBSO (145)

+ 62 more

NEW

JOKER & WITCH
Men Leather Straps Watch
Rs. 898 ~~Rs. 4699~~ (81% OFF)

JOKER & WITCH
Men Leather Straps Watch
Rs. 898 ~~Rs. 4699~~ (81% OFF)

IBSO
Sizes: Onesize
Rs. 920 ~~Rs. 5750~~ (84% OFF)

BLACK LOVIES
2 Pcs Tang Watch Straps
Rs. 491 ~~Rs. 2099~~ (84% OFF)

United Colors of Benetton
Men Analogue Watch UWUC...
Rs. 978 ~~Rs. 2445~~ (60% OFF)

UPTO ₹500 OFF

SAVED OUTFIT PAGE:

StyleSense

AI Fashion Stylist

Dashboard

Get Styled

Saved Outfits

History

Style Chat

Profile

Saved Outfits

Your favourite outfit recommendations, saved for later.

Casual College Look

10/6/2026

Budget-Friendly

Estimated Cost: ₹2,000

☆ The Burnt Orange pants complement the earthy tones in the patterned Black Brown T shirt, creating a harmonious color palette that suits the user's medium skin tone. The Teal Blue watch adds a pop of color, tying in with the cool tones in the shirt's pattern, making this outfit suitable for a college setting.

Your Base Item (Owned)

Brown T shirt — Black - patterned

Bottom

Slim fit pants — Burnt Orange - Cotton

ajio

amazon

Flipkart

Less details

Bottom: ₹1500 Accessories: ₹500

Accessories

Leather strap watch

ajio

amazon

Flipkart

RECOMMENDATION HISTORY PAGE:

StyleSense

AI Fashion Stylist

Dashboard

Get Styled

Saved Outfits

History

Style Chat

Profile

Style History

All your past styling sessions and outfit recommendations.

College ₹2000 - Male

10/6/2026, 7:23:28 pm - 3 outfits

Office ₹5000 - Male

10/6/2026, 7:15:18 pm - 3 outfits

Formal ₹5000 - Female

10/6/2026, 7:09:59 pm - 3 outfits

Formal ₹5000 - Female

10/6/2026, 7:09:21 pm - 3 outfits

Formal ₹5000 - Female

10/6/2026, 7:00:03 pm - 3 outfits

StyleSense

AI Fashion Stylist

Dashboard

Get Styled

Saved Outfits

History

Style Chat

Profile

Test user

Sign out

Style History

All your past styling sessions and outfit recommendations.

College ₹2000 · Male

10/6/2026, 7:23:28 pm · 3 outfits

Office ₹5000 · Male

10/6/2026, 7:15:18 pm · 3 outfits

Formal ₹5000 · Female

10/6/2026, 7:09:59 pm · 3 outfits

Style Advice:

For a deep skin tone, focus on vibrant colors like tangerine, fire red, and teal to create stunning contrasts. Balance these colors with neutral footwear and minimal accessories to complete the formal look.

View Color Palette

Tangerine Dreams

Balanced Estimated Cost: ₹3,500

This outfit complements the deep skin tone with the vibrant tangerine color, creating a stunning contrast. The formal occasion is

Fire Red Elegance

Premium Estimated Cost: ₹4,000

The fire red color is a perfect match for the deep skin tone, and the formal dress creates a stunning, elegant look suitable for a

Teal Sophistication

Balanced Estimated Cost: ₹3,000

The teal color complements the deep skin tone, creating a beautiful contrast. The formal top and trousers are perfect for a 20-

AI FASHION CHATBOT:

StyleSense

AI Fashion Stylist

Dashboard

Get Styled

Saved Outfits

History

Style Chat

Profile

Test user

Sign out

Style Chat

Ask your AI stylist anything about fashion and personal style.

Search chat history...

What colors suit my skin tone?

As someone with a deep skin tone, you'll look stunning in a variety of colors. Earthy shades like olive green, terracotta, and golden brown complement your skin beautifully. Rich jewel tones such as emerald green, sapphire blue, and ruby red also create a striking contrast. Additionally, soft pastels like peach, mint, and lavender can add a touch of warmth and softness to your look. In the Indian context, colors like maroon, teal, and mustard can also work well with your skin tone, especially during festivals and special occasions.

I'm StyleSense AI, a fashion-focused assistant. I can help with outfits, styling advice, fashion trends, color matching, and wardrobe recommendations. Please ask a fashion-related question.

who won ipl 2026

Ask about fashion, outfits, colors...

USER PROFILE:

StyleSense

AI Fashion Stylist

Dashboard

Get Styled

Saved Outfits

History

Style Chat

Profile

Test user

Sign out

Test user

test@gmail.com

Style Preferences

Gender

Male Female Non-binary Prefer not to say

Preferred Occasion

Casual College Office Party Formal Wedding

Age

Enter your age

Default Budget (₹)

₹500 ₹1,000 ₹2,000 ₹5,000 ₹10,000

2000

Save Preferences

CONCLUSION:

StyleSense is an AI-powered fashion recommendation system that delivers personalized outfit suggestions by combining Artificial Intelligence, Computer Vision, and modern web technologies. The application analyzes a user's skin tone and preferences using MediaPipe and OpenCV, then leverages the Groq Large Language Model (LLM) to generate customized outfit recommendations, suitable color palettes, and intelligent styling advice.

Developed with React.js, Flask, MongoDB, and JWT Authentication, the platform provides a secure, scalable, and user-friendly experience. Key features include personalized outfit recommendations, an AI fashion chatbot, recommendation history, saved outfits, and shopping suggestions. The successful integration of these technologies demonstrates the system's reliability and effectiveness in providing context-aware fashion guidance.

Overall, StyleSense simplifies fashion decision-making by helping users choose styles that complement their appearance and preferences. The project highlights the practical application of AI in the fashion industry and showcases the potential of intelligent recommendation systems to enhance user satisfaction and personalized digital experiences.